

Laporan Tugas Besar

Arion Compiler

IF2224 Teori Bahasa Formal dan Otomata



Oleh :
khusus 1352____

Daniel Anindito Nugroho	13524002
Ishak Palentino Napitupulu	13524022
Al Farabi	13524086
Ishaq Irfan Farizal	13524094

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2025

I. Teori Singkat

A. Bahasa Pemrograman dan Proses Kompilasi

Bahasa pemrograman adalah notasi formal yang digunakan manusia untuk mengekspresikan instruksi kepada komputer. CPU hanya mengetahui bahasa mesin, source code yang ditulis dalam bahasa pemrograman harus diterjemahkan terlebih dahulu. Proses ini dilakukan interpreter atau compiler.

Compiler menerjemahkan seluruh source code menjadi kode mesin sebelum program dijalankan, sedangkan interpreter memproses dan mengeksekusi source code baris per baris secara langsung. Keduanya memiliki tahapan : Lexical Analysis - Syntax Analysis - Semantic Analysis - Intermediate Code Generation.

B. Analisis Leksikal (Lexical Analysis)

Analisis leksikal adalah fase pertama dari sebuah compiler atau interpreter. Proses ini mengelompokkan deretan karakter dari input ke dalam unit token. Proses ini dilakukan Lexer. Selain menghasilkan token, analisis leksikal juga berfungsi untuk membuang bagian kode yang tidak berguna bagi sintaksis, seperti spasi kosong (whitespace), tab, dan komentar.

C. Token, Lexeme, dan Pattern

Token adalah kategori atau jenis dari suatu unit leksikal. Lexeme adalah rangkaian karakter aktual dalam source code yang cocok dengan suatu pola token. Pattern adalah aturan yang mendeskripsikan bentuk-bentuk yang valid untuk suatu token, biasanya dinyatakan sebagai ekspresi reguler (regular expression).

Setiap lexeme dikatakan valid jika memenuhi aturan yang telah didefinisikan oleh grammar bahasa melalui suatu pattern. Pattern menjelaskan apa yang dapat menjadi token, dan pattern-pattern ini didefinisikan menggunakan regular expression.

D. Deterministic Finite Automata (DFA)

Finite Automata adalah model yang terdiri dari sekumpulan state dan transisi untuk mengenali bahasa reguler. Deterministic Finite Automata (DFA), ada finite Automata dimana untuk setiap state dan setiap karakter input, hanya terdapat tepat satu transisi menuju state berikutnya.

DFA bekerja dengan membaca input karakter demi karakter. Jika setelah membaca seluruh leksem DFA berada pada final state, maka leksem tersebut dinyatakan valid sebagai token tertentu. Jika input tidak memiliki transisi yang valid atau berhenti di state yang bukan final state, maka Lexer akan mengeluarkan pesan kesalahan (misal token unknown). Setiap final state pada DFA merepresentasikan satu jenis token tertentu.

II. Perancangan & Implementasi

A. Penjelasan Umum Diagram Transisi DFA

DFA (Deterministic Finite Automata) yang dirancang untuk bahasa Arion ini berfungsi sebagai mesin pengenalan pola karakter tunggal menjadi satuan makna yang disebut Token. Secara arsitektural, DFA ini memulai setiap operasinya dari S0 (START).

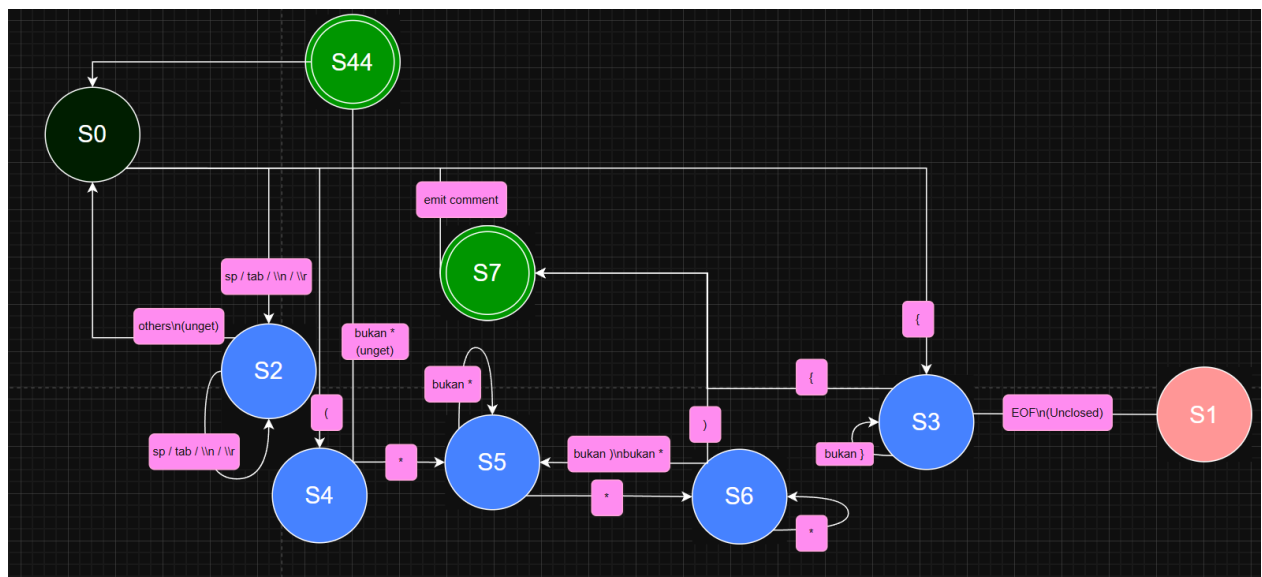
Mesin akan membaca input karakter demi karakter dan berpindah dari satu keadaan (state) ke keadaan lain berdasarkan aturan transisi yang telah didefinisikan.

Ciri utama dari DFA ini adalah penerapan strategi Greedy Matching (mencari pola terpanjang) dan mekanisme Lookahead menggunakan fungsi unget. DFA ini mampu menangani 52 jenis token hanya dengan sekitar 45 state berkat efisiensi pada bagian Identifier dan Keyword yang menggunakan tabel pencarian (lookup table). DFA juga dirancang untuk bersifat Robust melalui State S1 (unknown), di mana setiap karakter ilegal tidak akan menghentikan program, melainkan dilaporkan lokasinya (baris dan kolom) lalu dilewati untuk memproses token berikutnya.

B. Penjelasan Setiap Kategori Token

1. Kelompok Whitespace dan Komentar (S2, S3-S7)

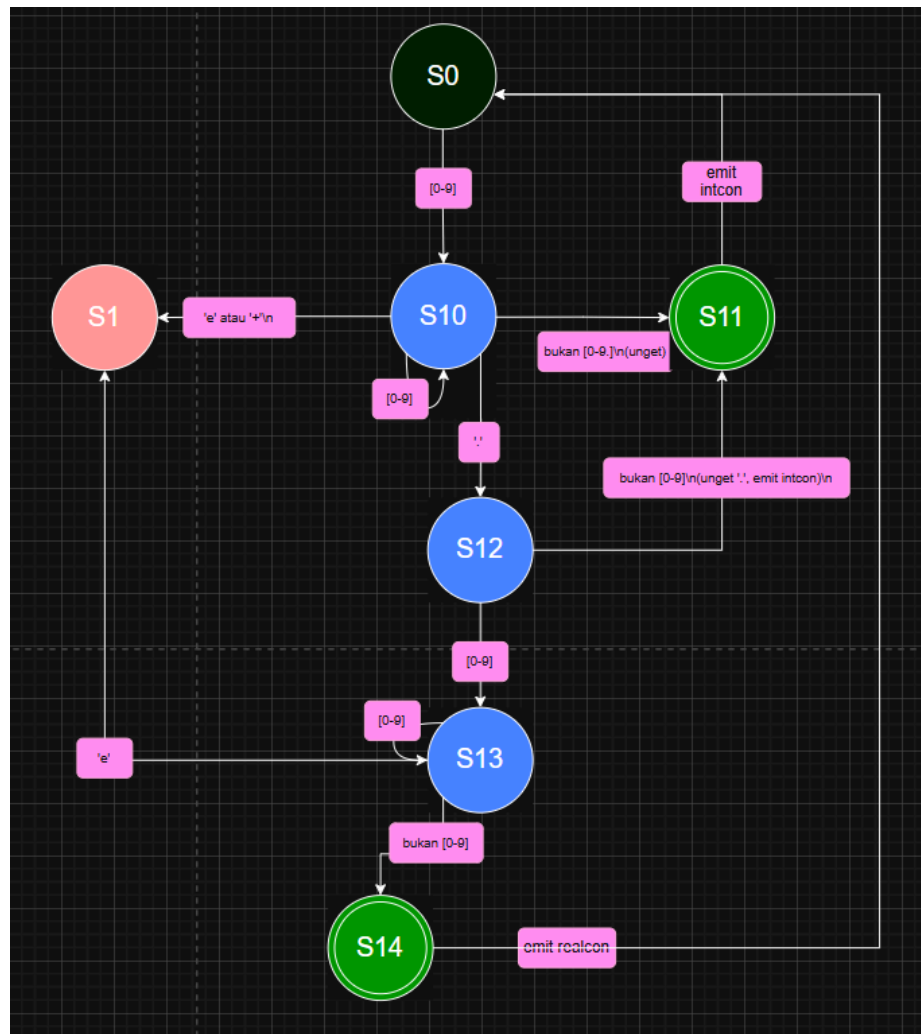
- Whitespace (S2): Karakter spasi (), tab (\t), dan pindah baris (\n, \r) diproses di State S2. Lexer akan terus berada di state ini selama membaca karakter whitespace dan akan melakukan reset ke S0 tanpa menghasilkan token apa pun.
- Komentar Gaya Kurung Kurawal (S3, S7): Ketika mendeteksi {, DFA masuk ke S3 dan mengabaikan semua karakter hingga menemukan }. Jika file berakhir (EOF) sebelum ditemukan }, DFA menuju S1 (unknown) karena komentar tidak tertutup.
- Komentar Gaya Bintang (S4, S5, S6): Kategori ini lebih kompleks karena simbol (bersifat ambigu. Di S4, Lexer melakukan lookahead:
 - Jika karakter berikutnya adalah *, maka resmi dianggap komentar gaya lama dan lanjut ke S5.
 - Jika karakter berikutnya bukan *, Lexer melakukan unget dan menghasilkan token lparent (kurung buka).
 - Penutupan komentar ini memerlukan urutan *) yang divalidasi melalui S6.



2. Kelompok Konstanta Numerik (S10 - S14)

DFA membedakan antara bilangan bulat dan bilangan riil melalui keberadaan titik desimal.

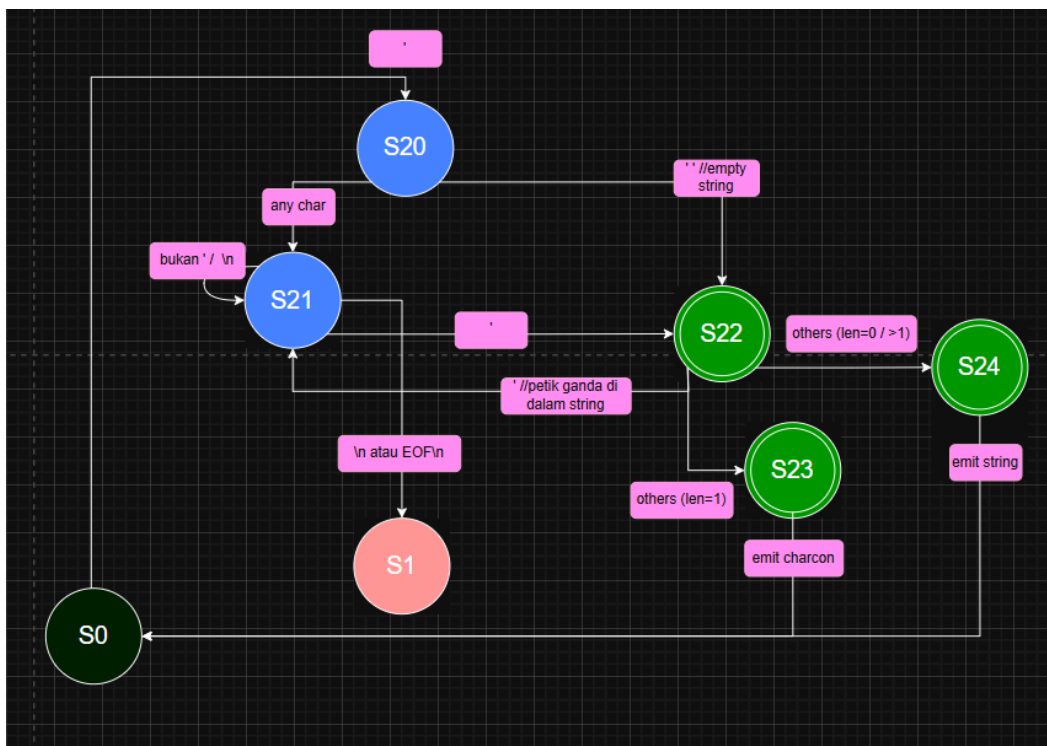
- Integer (S10, S11): Mengumpulkan digit 0-9. Jika karakter selanjutnya bukan titik atau angka, Lexer melakukan unget dan menghasilkan intcon.
- Real (S12, S13, S14): Jika ditemukan titik (.) di S10, mesin berpindah ke S12.
 - Aturan Khusus: Sesuai instruksi asisten, angka seperti 5. tidak dianggap sebagai bilangan riil yang valid. Jika di S12 karakter berikutnya bukan angka, titik tersebut akan dikembalikan (unget), dan angka 5 akan di-emit sebagai intcon.
 - Jika ditemukan angka di S12, maka mesin lanjut ke S13 untuk mengumpulkan bagian desimal dan menghasilkan realcon di S14.



3. Kelompok String dan Konstanta Karakter (S20 - S24)

Logika ini menangani teks yang diapit oleh petik tunggal (') dengan aturan ketat.

- a) Checkpoint Escape Character (S22): Ini adalah bagian paling krusial. Ketika sebuah petik ditemukan di tengah proses (S21 → S22), DFA tidak langsung berhenti. Mesin akan mengecek satu karakter lagi:
 - Jika ditemukan petik kedua ("), mesin menganggapnya sebagai satu karakter petik di dalam string dan kembali ke S21.
 - Jika ditemukan karakter lain, maka token dianggap selesai.
- b) Klasifikasi Akhir:
 - Jika jumlah karakter yang dikumpulkan tepat 1, di-emit sebagai charcon (S23).
 - Jika jumlah karakter 0 (string kosong) atau lebih dari 1, di-emit sebagai string (S24).
- c) Pelarangan Multiline: Jika ditemukan \n sebelum petik penutup, DFA akan pindah ke State S1 (emit Unknown).



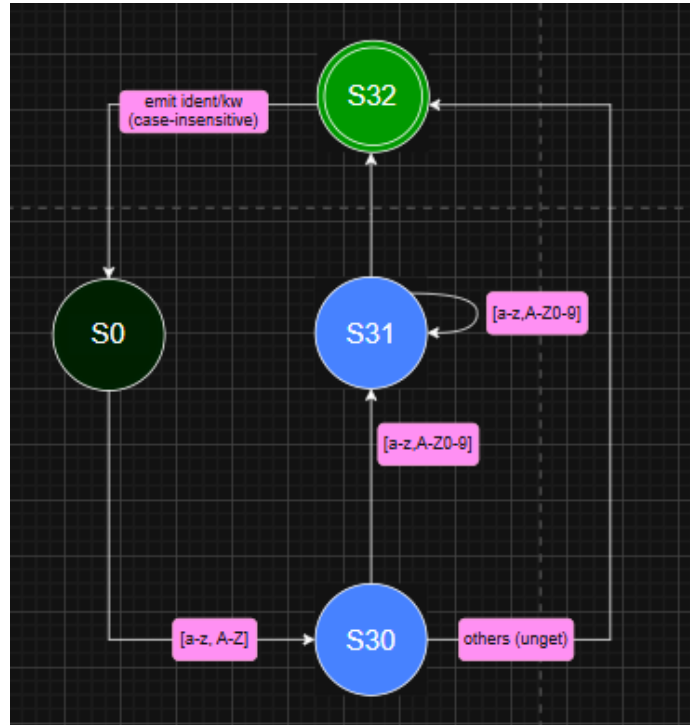
4. Kelompok Identifier dan Keywords (S30 - S32)

Kategori ini menangani penamaan variabel, fungsi, serta kata kunci cadangan bahasa Arion.

- a) Aturan Penamaan: Identifier harus dimulai dengan huruf. Karakter selanjutnya boleh kombinasi huruf dan angka. Penggunaan underscore () dilarang keras dan akan dianggap sebagai kesalahan leksikal jika muncul di State S0 atau S31.
- b) Efisiensi State S32: Dibandingkan membuat state untuk tiap keyword, DFA ini mengumpulkan semua kata ke dalam lexeme. Di State S32,

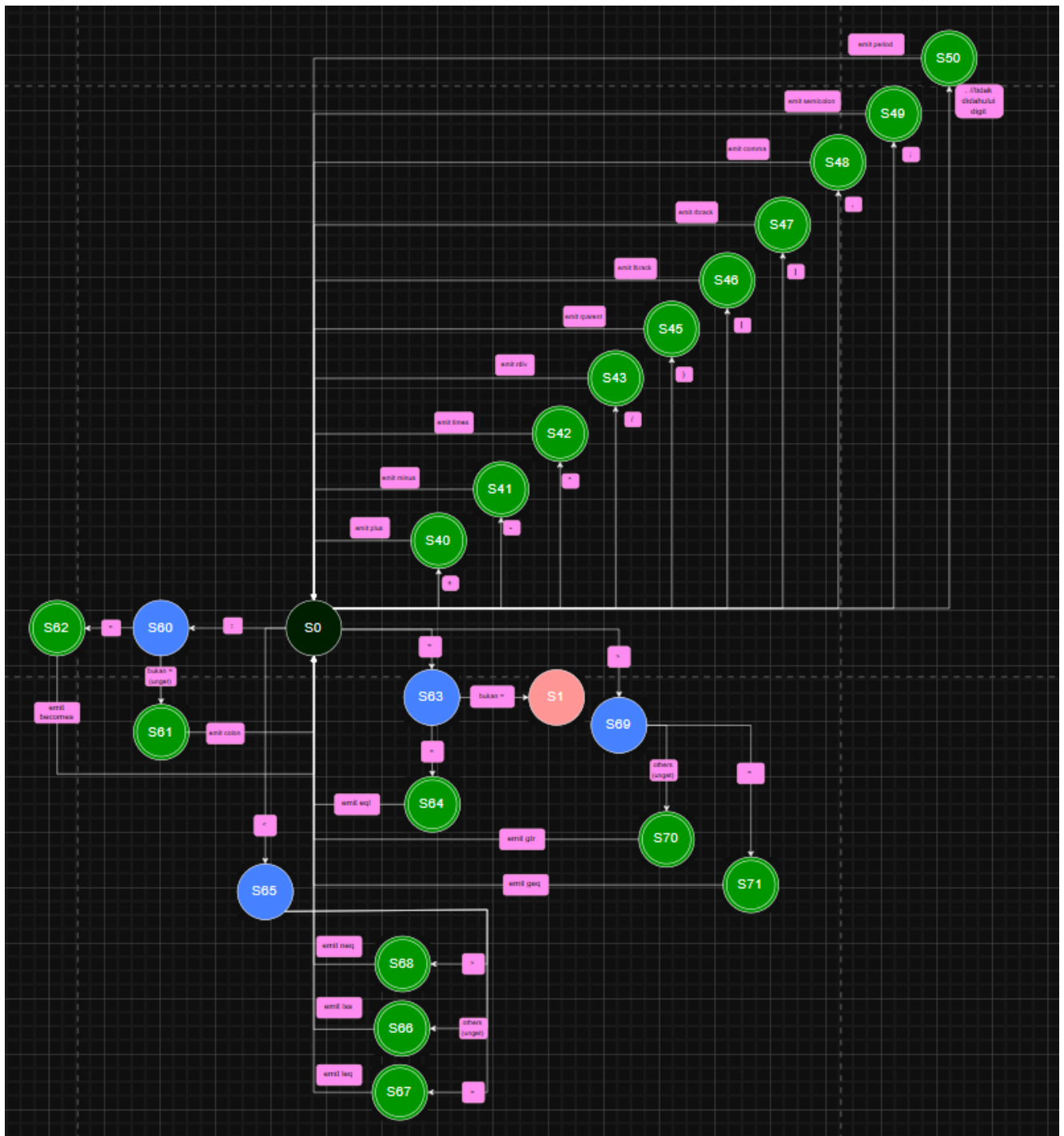
dilakukan pengecekan ke tabel referensi berisi 29 keyword (seperti begin, if, var).

- c) Case Insensitivity: Sebelum pengecekan, lexeme akan dikonversi ke huruf kecil (lowercase) untuk memastikan BEGIN, begin, dan BeGiN diidentifikasi sebagai token beginsy yang sama.



5. Kelompok Operator dan Simbol (S40 - S71)

- a) Operator Tunggal (S40 - S50): Simbol seperti +, -, *, /, [,], ,, ;, ., dan) bersifat deterministik langsung. Begitu dibaca dari S0, mereka langsung menuju Accept State masing-masing.
- b) Operator Kompleks (Lookahead):
 - Titik Dua (S60): Membedakan antara : (colon) dan := (becomes).
 - Sama Dengan (S63): simbol = tunggal adalah ilegal. DFA di S63 mewajibkan karakter berikutnya adalah = lagi untuk menjadi == (eq). Jika tidak, sistem akan melempar ke S1 (Unknown).
 - Perbandingan (S65, S69): Menangani logika tiga arah untuk simbol < (bisa menjadi <, <=, atau <>) dan dua arah untuk simbol > (bisa menjadi > atau >=).



6. Unknown Handling (S1)

State S1 berfungsi sebagai Accept State khusus untuk menangani masukan ilegal tanpa menghentikan proses pemindaian.

- Karakter tak dikenal (@, #, _), simbol = tunggal, atau token (string/komentar) yang tidak tertutup hingga akhir baris/file.

- Mencatat koordinat kesalahan.
- Menghasilkan (emit) token unknown.
- Melakukan reset kembali ke S0 (START) untuk memproses karakter selanjutnya.

C. Daftar Lengkap State

Kode	Tipe	Nama State	Keterangan / Trigger	Token
S0	Start	START	Titik awal setiap token; mengabaikan spasi/tab/newline.	—
S1	Error	UNKOWN_ACC	Menangani karakter ilegal; Menghasilkan token unkown	unknown
S2	Inter	WHITESPACE	Memproses karakter pemisah (space, tab, \n, \r).	—
S3	Inter	COMMENT_BRACE	Membaca isi komentar di dalam kurung kurawal {.	—
S4	Inter	COMMENT_PAR_1	Membaca (; mengecek apakah diikuti * atau tidak.	—
S5	Inter	COMMENT_PAR_2	Membaca isi komentar di dalam tanda (*.	—
S6	Inter	COMMENT_STAR	Membaca * di dalam komentar; mengecek penutup).	—
S7	Accept	COMMENT_END	Komentar berhasil ditutup; kembali ke S0.	comment
S10	Inter	INT_DIGIT	Akumulasi digit 0-9; jika bertemu e maka UNKNOWN.	—
S11	Accept	INTCON_ACC	Berhenti karena non-digit; melakukan unget karakter.	intcon
S12	Inter	REAL_DOT	Membaca .; jika non-digit maka unget titik ke S11.	—
S13	Inter	REAL_FRAC	Akumulasi digit di belakang titik desimal.	—
S14	Accept	REAL_ACC	Bagian desimal selesai; melakukan unget karakter.	realcon
S20	Inter	QUOTE_OPEN	Membaca ' pertama; jika diikuti ' lagi ke S22.	—
S21	Inter	STR_BODY	Akumulasi isi string; jika bertemu \n maka UNKNOWN.	—

S22	Inter	QUOTE_CHECK	Membaca ' ; jika ada ' lagi ke S21 (Escape).	—
S23	Accept	CHAR_ACC	Penutup ditemukan dan panjang isi = 1.	charcon
S24	Accept	STR_ACC	Penutup ditemukan dan panjang isi 0 atau > 1.	string
S30	Inter	IDENT_START	Karakter pertama berupa huruf [a-zA-Z].	—
S31	Inter	IDENT_CONT	Akumulasi huruf/angka; karakter _ dilarang).	—
S32	Accept	IDENT_KW_ACC	Melakukan lookup ke tabel 29 Keyword atau Identifier.	ident / kw
S40	Accept	PLUS_ACC	Membaca karakter +.	plus
S41	Accept	MINUS_ACC	Membaca karakter -.	minus
S42	Accept	TIMES_ACC	Membaca karakter *.	times
S43	Accept	RDIV_ACC	Membaca karakter /.	rdiv
S44	Accept	LPAR_ACC	Membaca (yang tidak diikuti oleh * (unget).	lparent
S45	Accept	RPAR_ACC	Membaca karakter).	rparent
S46	Accept	LBRK_ACC	Membaca karakter [.	lbrack
S47	Accept	RBRK_ACC	Membaca karakter].	rbrack
S48	Accept	COMMA_ACC	Membaca karakter ,.	comma
S49	Accept	SEMI_ACC	Membaca karakter ;.	semicolon
S50	Accept	DOT_ACC	Membaca karakter . (bukan bagian dari realcon).	period
S60	Inter	COLON_CHECK	Membaca ;; mengecek apakah diikuti =.	—
S61	Accept	COLON_ACC	Setelah S60, karakter bukan = (unget).	colon
S62	Accept	BECM_ACC	Setelah S60, karakter berikutnya =.	becomes
S63	Inter	EQL_CHECK	Membaca =; jika karakter berikut bukan = ke S1.	—
S64	Accept	EQL_ACC	Berhasil membaca urutan karakter ==.	eq
S65	Inter	LSS_CHECK	Membaca <; mengecek apakah diikuti = atau >.	—
S66	Accept	LSS_ACC	Setelah S65, karakter bukan = atau > (unget).	lss
S67	Accept	LEQ_ACC	Setelah S65, karakter berikutnya =.	leq

S68	Accept	NEQ_ACC	Setelah S65, karakter berikutnya >.	neq
S69	Inter	GTR_CHECK	Membaca >; mengecek apakah diikuti =.	—
S70	Accept	GTR_ACC	Setelah S69, karakter bukan = (unget).	gtr
S71	Accept	GEQ_ACC	Setelah S69, karakter berikutnya =.	geq
S99	Accept	EOF_ACC	Menandai akhir dari pembacaan file kode sumber.	eof

D. Implementasi Program

Implementasi program pada milestone ini berfokus pada pembangunan Lexical Analyzer berbasis Deterministic Finite Automata (DFA). Lexer membaca source code karakter demi karakter, menentukan perpindahan state, membentuk lexeme, lalu menghasilkan token ketika mencapai accept state.

Struktur Utama

- Lexer bertugas membaca file input dan menjalankan seluruh logika DFA.
- Token menyimpan hasil analisis leksikal berupa jenis token, lexeme, baris, dan kolom.
- LexerUtils menyediakan fungsi bantu seperti pengecekan huruf, digit, alfanumerik, dan konversi huruf kecil.
- main bertugas menerima file input, menjalankan proses tokenisasi, lalu menuliskan hasilnya ke file output.

Pseudocode

1. Struktur Data Token

```
struct Token
    type
    value
    line
    column
```

Setiap token yang dihasilkan lexer disimpan dalam struktur Token. Selain jenis token dan lexeme, posisi baris dan kolom juga dicatat agar kesalahan leksikal dapat diidentifikasi dengan tepat

2. Inisiasi Lexer

```
function Lexer(filename)
    source_file <- open(filename)
    line <- 1
```

```

column <- 1
unget_buffer <- empty_stack
next_number_negative <- false

if source_file is invalid
    print "Lexical Error: Cannot open source file"

```

Saat objek lexer dibuat, file sumber dibuka lebih dahulu. Program juga menyiapkan penghitung baris dan kolom, buffer untuk lookahead, serta flag `next_number_negative`.

3. Membaca dan Mengembalikan Karakter

```

function ReadChar()
    if unget_buffer is not empty
        c <- pop from unget_buffer
    else
        c <- read one character from source_file

    if c is not EOF
        if c = newline
            line <- line + 1
            column <- 1
        else
            column <- column + 1

    return c

function UnreadChar(c, prev_line, prev_col)
    if c is not EOF
        push c into unget_buffer
        line <- prev_line
        column <- prev_col

```

`ReadChar()` membaca karakter berikutnya, baik dari file maupun dari `unget_buffer`. `UnreadChar()` dipakai untuk mengembalikan karakter yang sudah terbaca ketika karakter tersebut ternyata menjadi awal token berikutnya. Mekanisme ini digunakan untuk lookahead dan longest match.

4. Kerangka Utama DFA

```

function GetNextToken()
    if source_file is invalid
        return EOF token

    currentState <- S0

```

```

currentLexeme <- ""
start_line <- line
start_col <- column

while true
  prev_line <- line
  prev_col <- column
  c <- ReadChar()

  switch currentState

    case S0:
      start_line <- prev_line
      start_col <- prev_col
      currentLexeme <- ""

      if c = EOF
        currentState <- S99

      else if c is whitespace
        currentState <- S2

      else if c = '{'
        currentState <- S3
        currentLexeme <- currentLexeme + c

      else if c = '('
        currentState <- S4
        currentLexeme <- currentLexeme + c

      else if c is digit
        currentState <- S10
        if next_number_negative = true
          currentLexeme <- currentLexeme +
'_'

          next_number_negative <- false
          currentLexeme <- currentLexeme + c

      else if c = '"'
        currentState <- S20
        currentLexeme <- currentLexeme + c

      else if c is alphabet
        currentState <- S30
        currentLexeme <- currentLexeme + c

      else if c = '+'
        currentState <- S40
        currentLexeme <- currentLexeme + c

      else if c = '-'
        currentState <- S41
        currentLexeme <- currentLexeme + c

      else if c = '*'
        currentState <- S42

```

```

        currentLexeme <- currentLexeme + c

    else if c = '/'
        currentState <- S43
        currentLexeme <- currentLexeme + c

    else if c = ')'
        currentState <- S45
        currentLexeme <- currentLexeme + c

    else if c = '['
        currentState <- S46
        currentLexeme <- currentLexeme + c

    else if c = ']'
        currentState <- S47
        currentLexeme <- currentLexeme + c

    else if c = ','
        currentState <- S48
        currentLexeme <- currentLexeme + c

    else if c = ';'
        currentState <- S49
        currentLexeme <- currentLexeme + c

    else if c = '.'
        currentState <- S50

    else if c = ':'
        currentState <- S60
        currentLexeme <- currentLexeme + c

    else if c = '='
        currentState <- S63
        currentLexeme <- currentLexeme + c

    else if c = '<'
        currentState <- S65
        currentLexeme <- currentLexeme + c

    else if c = '>'
        currentState <- S69
        currentLexeme <- currentLexeme + c

    else
        currentState <- S1
        currentLexeme <- currentLexeme + c

case S1:
    print lexical error with currentLexeme,
start_line, start_col
    UnreadChar(c, prev_line, prev_col)
    return TOKEN_ERROR

case S99:

```

```
UnreadChar(c, prev_line, prev_col)
return TOKEN_EOF
```

Bagian ini merupakan inti implementasi DFA. State awal adalah S0, lalu program berpindah ke state lain sesuai karakter pertama yang dibaca. Jika input tidak valid, lexer masuk ke S1. Jika file berakhir, lexer masuk ke S99.

5. Whitespace dan Komentar

```
case S2:
  if c is whitespace
    stay in S2
  else
    UnreadChar(c, prev_line, prev_col)
    currentState <- S0

case S3:
  if c = EOF
    currentState <- S1
  else if c = '}'
    currentState <- S7
  else
    currentState <- S3
    currentLexeme <- currentLexeme + c

case S4:
  if c = '*'
    currentState <- S5
  else
    UnreadChar(c, prev_line, prev_col)
    currentState <- S44
    currentLexeme <- "("

case S5:
  if c = EOF
    currentState <- S1
  else if c = '*'
    currentState <- S6
  else
    currentState <- S5
    currentLexeme <- currentLexeme + c

case S6:
  if c = EOF
    currentState <- S1
  else if c = ')'
    currentState <- S7
  else if c = '*'
    currentState <- S6
    currentLexeme <- currentLexeme + '*'
```

```

else
    currentState <- S5
    currentLexeme <- currentLexeme + '*'
    currentLexeme <- currentLexeme + c

case S7:
    UnreadChar(c, prev_line, prev_col)
    return TOKEN_COMMENT

```

Whitespace diabaikan dan tidak menghasilkan token. Komentar didukung dalam dua bentuk, yaitu { ... } dan (* ... *). Setelah komentar selesai dibaca, program tetap mengembalikannya sebagai token comment.

6. Bilangan Integer dan Real

```

case S10:
    if c is digit
        currentState <- S10
        currentLexeme <- currentLexeme + c
    else if c = '.'
        currentState <- S12
        currentLexeme <- currentLexeme + c
    else
        UnreadChar(c, prev_line, prev_col)
        currentState <- S11

case S11:
    UnreadChar(c, prev_line, prev_col)
    return TOKEN_INTCON

case S12:
    if c is digit
        currentState <- S13
        currentLexeme <- currentLexeme + c
    else
        UnreadChar(c, prev_line, prev_col)
        UnreadChar('.', start_line, start_col +
length(currentLexeme) - 1)
        remove last character from currentLexeme
        currentState <- S11

case S13:
    if c is digit
        currentState <- S13
        currentLexeme <- currentLexeme + c
    else if c = 'e'
        currentState <- S1
        currentLexeme <- currentLexeme + c
    else
        UnreadChar(c, prev_line, prev_col)
        currentState <- S14

```

```
case S14:
    UnreadChar(c, prev_line, prev_col)
    return TOKEN_REALCON
```

Bilangan diproses mulai dari S10. Jika setelah digit ditemukan titik dan sesudahnya masih ada digit, token diklasifikasikan sebagai realcon. Jika titik tidak diikuti digit, titik dikembalikan ke buffer dan token diperlakukan sebagai intcon.

7. String dan Karakter Constant

```
case S20:
    if c = '"'
        currentState <- S22
        currentLexeme <- currentLexeme + c
    else if c = newline or c = EOF
        currentState <- S1
        if c is not EOF
            currentLexeme <- currentLexeme + c
    else
        currentState <- S21
        currentLexeme <- currentLexeme + c

case S21:
    if c = newline or c = EOF
        currentState <- S1
        if c is not EOF
            currentLexeme <- currentLexeme + c
    else if c = '"'
        currentState <- S22
        currentLexeme <- currentLexeme + c
    else
        currentState <- S21
        currentLexeme <- currentLexeme + c

case S22:
    if c = '"'
        currentState <- S21
        currentLexeme <- currentLexeme + c
    else
        UnreadChar(c, prev_line, prev_col)

        actual_len <- 0
        for i <- 1 to length(currentLexeme) - 2
            if currentLexeme[i] = '"' and currentLexeme[i +
1] = '"'
                i <- i + 1
                actual_len <- actual_len + 1

        if actual_len = 1
            currentState <- S23
```

```

        else
            currentState <- S24

case S23:
    UnreadChar(c, prev_line, prev_col)
    return TOKEN_CHARCON

```

String dan karakter diproses dari lexeme yang diapit petik tunggal. Setelah petik penutup ditemukan, program menghitung panjang isi aktual. Jika panjangnya satu, token menjadi charcon; selain itu menjadi string. Jika newline atau EOF muncul sebelum penutup, token dianggap error.

8. Identifier dan Keyword

```

case S30:
    if c is alphanumeric
        currentState <- S31
        currentLexeme <- currentLexeme + c
    else
        UnreadChar(c, prev_line, prev_col)
        currentState <- S32

case S31:
    if c is alphanumeric
        currentState <- S31
        currentLexeme <- currentLexeme + c
    else
        UnreadChar(c, prev_line, prev_col)
        currentState <- S32

case S32:
    UnreadChar(c, prev_line, prev_col)
    lower_ident <- to_lowercase(currentLexeme)

    if lower_ident = "not"           return TOKEN_NOTSY
    if lower_ident = "div"           return TOKEN_IDIV
    if lower_ident = "mod"           return TOKEN_IMOD
    if lower_ident = "and"           return TOKEN_ANDSY
    if lower_ident = "or"            return TOKEN_ORSY
    if lower_ident = "const"         return TOKEN_CONSTSY
    if lower_ident = "type"          return TOKEN_TYPESY
    if lower_ident = "var"           return TOKEN_VARSY
    if lower_ident = "function"      return TOKEN_FUNCTIONSY
    if lower_ident = "procedure"     return TOKEN_PROCEDURESY
    if lower_ident = "array"         return TOKEN_ARRAYSY
    if lower_ident = "record"        return TOKEN_RECORDSY
    if lower_ident = "program"       return TOKEN_PROGRAMSY
    if lower_ident = "begin"         return TOKEN_BEGINSY
    if lower_ident = "if"            return TOKEN_IFSY
    if lower_ident = "case"          return TOKEN_CASESY
    if lower_ident = "repeat"        return TOKEN_REPEATSY

```

```

if lower_ident = "while"      return TOKEN_WHILESY
if lower_ident = "for"       return TOKEN_FORSY
if lower_ident = "end"       return TOKEN_ENDSY
if lower_ident = "else"      return TOKEN_ELSESY
if lower_ident = "until"     return TOKEN_UNTILSY
if lower_ident = "of"        return TOKEN_OFSY
if lower_ident = "do"        return TOKEN_DOSY
if lower_ident = "to"        return TOKEN_TOSY
if lower_ident = "downto"    return TOKEN_DOWNTOSY
if lower_ident = "then"      return TOKEN_THENSY

return TOKEN_IDENT

```

Identifier harus diawali huruf. Setelah lexeme selesai dibaca, program mengubahnya ke huruf kecil lalu membandingkannya satu per satu dengan daftar keyword. Jika tidak ada kecocokan, token diklasifikasikan sebagai ident.

9. Operator dan Simbol

```

case S40:
    UnreadChar(c, prev_line, prev_col)
    return TOKEN_PLUS

case S41:
    UnreadChar(c, prev_line, prev_col)

    if c is digit
        next_number_negative <- true

    return TOKEN_MINUS

case S42:
    UnreadChar(c, prev_line, prev_col)
    return TOKEN_TIMES

case S43:
    UnreadChar(c, prev_line, prev_col)
    return TOKEN_RDIV

case S44:
    UnreadChar(c, prev_line, prev_col)
    return TOKEN_LPARENT

case S45:
    UnreadChar(c, prev_line, prev_col)
    return TOKEN_RPARENT

case S46:
    UnreadChar(c, prev_line, prev_col)
    return TOKEN_LBRACK

case S47:
    UnreadChar(c, prev_line, prev_col)

```

```

        return TOKEN_RBRACK

case S48:
    UnreadChar(c, prev_line, prev_col)
    return TOKEN_COMMA

case S49:
    UnreadChar(c, prev_line, prev_col)
    return TOKEN_SEMICOLON

case S50:
    UnreadChar(c, prev_line, prev_col)
    return TOKEN_PERIOD

case S60:
    if c = '='
        currentState <- S62
        currentLexeme <- currentLexeme + c
    else
        UnreadChar(c, prev_line, prev_col)
        currentState <- S61

case S61:
    UnreadChar(c, prev_line, prev_col)
    return TOKEN_COLON

case S62:
    UnreadChar(c, prev_line, prev_col)
    return TOKEN_BECOMES

case S63:
    if c = '='
        currentState <- S64
        currentLexeme <- currentLexeme + c
    else
        UnreadChar(c, prev_line, prev_col)
        currentState <- S1

case S64:
    UnreadChar(c, prev_line, prev_col)
    return TOKEN_EQL

case S65:
    if c = '='
        currentState <- S67
        currentLexeme <- currentLexeme + c
    else if c = '>'
        currentState <- S68
        currentLexeme <- currentLexeme + c
    else
        UnreadChar(c, prev_line, prev_col)
        currentState <- S66

case S66:
    UnreadChar(c, prev_line, prev_col)
    return TOKEN_LSS

```

```

case S67:
  UnreadChar(c, prev_line, prev_col)
  return TOKEN_LEQ

case S68:
  UnreadChar(c, prev_line, prev_col)
  return TOKEN_NEQ

case S69:
  if c = '='
    currentState <- S71
    currentLexeme <- currentLexeme + c
  else
    UnreadChar(c, prev_line, prev_col)
    currentState <- S70

case S70:
  UnreadChar(c, prev_line, prev_col)
  return TOKEN_GTR

case S71:
  UnreadChar(c, prev_line, prev_col)
  return TOKEN_GEQ

```

Operator tunggal seperti +, -, *, /, (,), [,], ,, :, dan . diproses melalui accept state masing-masing. Operator majemuk seperti :=, ==, <=, <>, dan >= memerlukan lookahead satu karakter. Pada implementasi ini, = tunggal dianggap tidak valid dan akan menghasilkan error.

10. Tokenisasi Keseluruhan

```

function Tokenize()
  tokens <- empty list

  while true
    t <- GetNextToken()
    append t to tokens

    if t.type = TOKEN_EOF
      break

  return tokens

```

Fungsi tokenize() memanggil get_next_token() berulang kali sampai token EOF diperoleh. Seluruh token, termasuk EOF, disimpan dalam vector hasil tokenisasi.

III. Pengujian

No	Test Case / Input / Output
1	Test Case 1 : tes semua token yang ada

test > milestone-1 > ≡ ti1.txt

```
1  program AllTokens;
2  const PI := 3.14;
3  type MyChar := 'k';
4  var
5      x, y: Integer;
6      arr: array;
7      rec: record
8  begin
9      x := 5;
10     y := x + 10;
11     if x == y then
12     |   x := x - 1
13     else
14     |   x := x * 2;
15     if x <> y then
16     |   y := x div 2;
17     y := x mod 3;
18     y := x / 2;
19     if x > y then
20     |   x := x + 1;
21     if x >= y then
22     |   x := x - 1;
23     if x < y then
24     |   x := x + 1;
25     if x <= y then
26     |   x := x - 1;
27     while x > 0 do
28     |   x := x - 1;
29     for x := 1 to 10 do
30     |   y := y + 1;
31     for x := 10 downto 1 do
32     |   y := y - 1;
33     repeat
34     |   x := x + 1;
35     until x == 5;
36     case x of
37     |   1: y := 1;
38     end;
39     if NOT (x == 0) AND (y <> 0) OR (x > y) then
40     |   x := 1;
41     { This is a comment }
42     (* This is also a comment *)
43     x := y;
44 end.
```

test > milestone-1 > ≡ ti1.txt_OUTPUT.txt

```
1  programsy
2  ident (AllTokens)
3  semicolon
4  constsy
5  ident (PI)
6  becomes
7  realcon (3.14)
8  semicolon
9  typesy
10 ident (MyChar)
11 becomes
12 charcon ('k')
13 semicolon
14 varsy
15 ident (x)
16 comma
17 ident (y)
18 colon
19 ident (Integer)
20 semicolon
21 ident (arr)
22 colon
23 arraysy
24 semicolon
25 ident (rec)
26 colon
27 recordsy
28 beginsy
29 ident (x)
30 becomes
31 intcon (5)
32 semicolon
33 ident (y)
34 becomes
35 ident (x)
36 plus
37 intcon (10)
38 semicolon
39 ifsy
40 ident (x)
41 eql
42 ident (y)
43 thensy
44 ident (x)
45 becomes
46 ident (x)
47 minus
```

2	Test Case 2: tes case insensitive <pre>test > milestone-1 > ≡ ti2.txt 1 iF NoT MoD aNd Or 2 IF NOT MOD AND OR 3 if not mod and or 4 arrayzz 5 array zz 6 forsy2 7 ident123 8 program</pre>

	<pre> test > milestone-1 > ≡ ti2.txt_OUTPUT.txt 1 ifsy 2 notsy 3 imod 4 andsy 5 orsy 6 ifsy 7 notsy 8 imod 9 andsy 10 orsy 11 ifsy 12 notsy 13 imod 14 andsy 15 orsy 16 ident (arrayzz) 17 arraysy 18 ident (zz) 19 ident (forsy2) 20 ident (ident123) 21 programsy 22 </pre>
3	Test Case 3: Tes edge case untuk intcon, realcon, period, plus, dan minus

```
test > milestone-1 > ≡ ti3.txt
```

```
1  0123
2  3.14
3  26.7
4  +67
5  1+67
6  -67
7  6-7
8  0.18
9  5.
10 .18
11 192.168.10.1
12 1.2e-10
13 123N
```

	<pre>test > milestone-1 > ≡ ti3.txt_OUTPUT.txt 1 intcon (0123) 2 realcon (3.14) 3 realcon (26.7) 4 plus 5 intcon (67) 6 intcon (1) 7 plus 8 intcon (67) 9 minus 10 intcon (-67) 11 intcon (6) 12 minus 13 intcon (-7) 14 realcon (0.18) 15 intcon (5) 16 period 17 period 18 intcon (18) 19 realcon (192.168) 20 period 21 realcon (10.1) 22 unknown (1.2e) 23 minus 24 intcon (-10) 25 intcon (123) 26 ident (N) 27</pre>
4	Test Case 4: Tes charcon, string, escape char, empty string, dan multiline string error.

	<pre> test > milestone-1 > ≡ ti4.txt 1 'j' 2 'k' 3 'IRK' 4 'TBFO' 5 '' 6 '''' 7 ''' 8 'baca_buku' 9 'Result = ' 10 'string\n''' </pre>
	<pre> test > milestone-1 > ≡ ti4.txt_OUTPUT.txt 1 charcon ('j') 2 charcon ('k') 3 string ('IRK') 4 string ('TBFO') 5 string ('') 6 charcon ('''') 7 unknown (''' 8) 9 string ('baca_buku') 10 string ('Result =') 11 string ('string\n''') 12 </pre>
5	Test Case 5: tes token komen, multiline, nested symbol, brackets dan comment error.

```
test > milestone-1 > ≡ ti5.txt
```

```
1 { This is a single-line comment }  
2 (* This is also a comment *)  
3 {  
4 Multiline  
5 comment here  
6 }  
7 (*  
8 Another  
9 multiline  
10 comment  
11 *)  
12 (.....)  
13 (* Atau {gini )  
14 { Atau gini )}
```

```
test > milestone-1 > ≡ ti5.txt_OUTPUT.txt
1  comment ( This is a single-line comment )
2  comment (( This is also a comment )
3  comment (
4  Multiline
5  comment here
6  )
7
8  comment ((
9  Another
10 multiline
11 comment
12 )
13
14 lparent
15 period
16 period
17 period
18 period
19 period
20 rparent
21 unknown (( Atau {gini )
22 { Atau gini ))
23
```

6	Test Case 6: tes simbol perbandingan dan aritmatika
---	---

test > milestone-1 > ≡ ti6.txt

```
1    ==
2    :=
3    =
4    <>
5    <
6    <=
7    >
8    >=
9    +
10   -
11   *
12   /
13   :
14   ;
15   ,
16   .
17   [
18   ]
19   (
20   )
```

	<pre> test > milestone-1 > ≡ ti6.txt_OUTPUT.txt 1 eql 2 becomes 3 unknown (=) 4 neq 5 lss 6 leq 7 gtr 8 geq 9 plus 10 minus 11 times 12 rdiv 13 colon 14 semicolon 15 comma 16 period 17 lbrack 18 rbrack 19 lparent 20 rparent 21 </pre>
7	Test Case 7: tes error handler lexer <pre> test > milestone-1 > ≡ ti7.txt 1 x := 5; 2 @ invalid symbol here 3 y := x + 10; 4 baca_buku 5 nama+ 6 = 7 x := y; </pre>

	<pre>test > milestone-1 > ≡ ti7.txt_OUTPUT.txt 1 ident (x) 2 becomes 3 intcon (5) 4 semicolon 5 unknown (@) 6 ident (invalid) 7 ident (symbol) 8 ident (here) 9 ident (y) 10 becomes 11 ident (x) 12 plus 13 intcon (10) 14 semicolon 15 ident (baca) 16 unknown (_) 17 ident (buku) 18 ident (nama) 19 plus 20 unknown (=) 21 ident (x) 22 becomes 23 ident (y) 24 semicolon 25 </pre>
8	Test Case 8: tes file handler <pre>test > milestone-1 > ≡ ti8.txt 1 █</pre>

test > milestone-1 > ≡ ti8.txt_OUTPUT.txt

1

IV. Kesimpulan dan Saran

Berdasarkan hasil perancangan dan implementasi pada Milestone 1, dapat disimpulkan bahwa Lexical Analyzer untuk bahasa pemrograman Arion telah berhasil dikembangkan dengan menggunakan prinsip Deterministic Finite Automata (DFA). Program ini terbukti mampu mentransformasikan aliran karakter dari kode sumber menjadi rangkaian satuan makna (token) yang terstruktur, mencakup seluruh 52 jenis token spesifikasi serta satu tambahan token unknown untuk menangani karakter ilegal. Melalui penerapan mekanisme lookahead dan fungsi unget, DFA secara akurat mampu memproses kasus-kasus kritis seperti pembedaan operator relasional, validasi angka riil, serta penanganan escape character pada konstanta string. Secara keseluruhan, Lexer ini memiliki sifat robust karena mampu melakukan pemulihan kesalahan leksikal (Error Recovery) sehingga proses pemindaian tetap dapat berlangsung hingga akhir file tanpa mengalami kegagalan sistem.

Untuk pelaksanaan tugas besar di masa mendatang, disarankan agar spesifikasi awal yang diberikan kepada mahasiswa dapat disusun secara lebih detail dan mendalam guna meminimalisir munculnya ambiguitas pada kasus-kasus leksikal yang kompleks. Penjelasan spesifikasi yang lebih rinci sejak awal rilis milestone diharapkan dapat mengurangi frekuensi revisi aturan atau tambahan instruksi, sehingga setiap kelompok memiliki panduan teknis yang lebih pasti, konsisten, dan komprehensif dalam merancang logika DFA maupun implementasi kode program sejak tahap awal pengerjaan.

V. Lampiran

A. Link Repository Github

<https://github.com/alfarabi0642/PRX-Tubes-IF2224-2026>

B. Link Workspace Diagram Transisi DFA

https://drive.google.com/file/d/1P9Z9j7my_zzqvfgHmYJj59elzCZ_Gwz/view?usp=sharing

C. Pembagian Tugas

NIM	Tugas
13524002	Pembuatan laporan, perancangan DFA, penggambaran DFA, implementasi utils.cpp dan utils.hpp
13524022	Pembuatan laporan, perancangan DFA, penggambaran DFA, implementasi main.cpp, MakeFile, dan readme
13524086	Pembuatan laporan, perancangan DFA, penggambaran DFA,

	implementasi lexer.cpp dan lexer.hpp
13524094	Pembuatan laporan, perancangan DFA, penggambaran DFA, pengetesan program, implementasi token.cpp dan token.hpp

VI. Referensi

1. Tutorialspoint. "Compiler Design - Lexical Analysis."
https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.html
2. J. E. Hopcroft, R. Motwani, dan J. D. Ullman. Introduction to Automata Theory, Languages, and Computation, 3rd ed.
https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613_Introduction-to-Automata-Theory,-Languages,-and-Computation-Edition-3.pdf
3. A. V. Aho, M. S. Lam, R. Sethi, dan J. D. Ullman. Compilers: Principles, Techniques, and Tools, 2nd ed. (Dragon Book), 2006.
[https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf)